

Start Here

How to use the Automotive Academy app & docs

"You don't learn to drive by reading the manual. You learn by driving."

- run the app, break things, read the notes
- every concept has a live animation
- interview answers are in book 04

A Head First-style learning guide that goes with the **Automotive Academy** sample app (Android Automotive OS, Kotlin, Jetpack Compose). Generated September 2026.

What's in this book

1 Welcome to Automotive Academy	3
2 Set up your garage	4
3 The lesson map	5
4 A 14-day learning plan	6
5 Regenerating these PDFs	7

1 Welcome to Automotive Academy

why this book looks different

Your brain is a clever machine: it ignores anything boring and keeps anything that looks important, surprising or *fun*. Textbooks full of dense paragraphs get ignored. So these docs are written **Head First style**: short pieces, pictures, questions to make you think, and the same idea repeated in different ways.

There are five PDFs. Each one has a single job:

Book	What it is for	Read it when...
00 Start Here	Set up, run the app, the learning plan	Day 1
01 Project Structure	Every folder, file and design decision of the sample app	You open the code
02 Automotive Concepts	AAOS, VHAL, CAN, HVAC, UX restrictions, media, audio, power, OTA, EVS, ...	Every day, one chapter
03 Automotive Security	Threats, standards (ISO/SAE 21434, UNECE R155/R156), secure boot, SecOC, keys	After concepts
04 Interview Q&A;	Sourced company questions + a big AAOS topic bank with answers	Last 7 days before interviews

BRAIN POWER

Before reading further: can you say, in one sentence, the difference between **Android Auto** and **Android Automotive OS**? Write it down. You will check your answer in book 02, chapter 1.

The signature boxes you will see everywhere:

Box	Meaning
BRAIN POWER	Stop and think. No answer given right away; the thinking is the learning.
THERE ARE NO DUMB QUESTIONS	The questions everyone is afraid to ask in an interview.
SHARPEN YOUR PENCIL	A small exercise. Answers are at the end of the chapter.
WATCH IT!	A common mistake or trap. Interviewers love these.
REAL LIFE	Where the concept is used in real cars you can buy today.
SEE IT MOVE: IN THE APP	Which screen of the app animates this concept, and what to try.
BULLET POINTS	The chapter summary. Reread these the night before an interview.
FIRESIDE CHAT	Two concepts argue with each other, so you remember the difference.

2 Set up your garage

tools you need

- **Android Studio** (Narwhal or newer) with **JDK 17+**. The project uses AGP 8.13, Kotlin 2.2, Gradle 8.13.
- **Android SDK Platform 36**. It ships `optional/android.car.jar`, which is what `useLibrary("android.car")` compiles against.
- **An AAOS emulator image** (SDK Manager → SDK Platforms → Show Package Details → "Android Automotive with Google APIs" / "Automotive Distant Display"). Optional but highly recommended.
- **Desktop Head Unit (DHU)** (SDK Tools → Android Auto Desktop Head Unit Emulator) to try the Car App Library service through Android Auto.
- Python 3 + reportlab only if you want to regenerate these PDFs.

Build and test from the terminal:

```
./gradlew :app:assembleDebug      # -> app/build/outputs/apk/debug/app-debug.apk
./gradlew :app:testDebugUnitTest  # 17 JVM unit tests (CAN, SecOC, VHAL IDs, ...)
adb install -r app/build/outputs/apk/debug/app-debug.apk

# jump straight to a lesson (routes are listed in chapter 3)
adb shell am start -n com.deepak.automotive/.MainActivity --es lesson can
```

WATCH IT!

The **same APK** runs on a phone *and* on AAOS. On a phone, the app uses its built-in vehicle **simulator**. On an AAOS emulator you can switch to **Real Car API** on the home screen, and the numbers then come from the emulator's VHAL (Extended controls → Car data).

On AAOS a third-party app gets a **SecurityException** when it tries to change HVAC. That is expected: `CONTROL_CAR_CLIMATE` is a signature|privileged permission. The app shows the error on screen on purpose, because this is a favourite interview question.

3 The lesson map

screen → concept → animation

#	Lesson (route)	Animation to watch	Concept
1	AAOS Architecture (architecture)	A request packet travels down the 6 layers and the event travels back up	App → Car API → CarService → VHAL → MCU → ECUs
2	Instrument Cluster (cluster)	Spring-animated needles, gear slide, blinking indicators, overspeed tell-tale	Cluster vs IVI, safety
3	Vehicle HAL (vhal)	Rows flash on every change event; hex property ID splits into coloured fields	Property IDs, change modes, areas
4	HVAC (hvac)	Fan speed drives blade rotation and air particles; temperature colour moves blue → red	Area IDs, privileged permissions
5	Driver Distraction (uxr)	Restriction chips bounce in; video gets locked; lists collapse; text truncates	CarUxRestrictions
6	Media (media)	Equalizer bars, track title slides in, seek bar extrapolated from PlaybackState	MediaBrowserService / MediaSession
7	Car Audio (audio)	Mixer bars duck or mute when a nav prompt / call arrives	Audio focus matrix
8	CAN Bus (can)	Frames travel along CAN_H/CAN_L; bit-by-bit arbitration reveals the winner	IDs, priority, DBC signals
9	Park Assist (park)	Ultrasonic waves expand toward the wall; colour zones; beep interval	Sensors, EVS, FMVSS 111
10	EV & Charging (ev)	Battery fills with a pulsing bolt; dot moves along the CC/CV charging curve	EV properties, charging
11	Power (power)	Active state pulses; transitions logged	CPMS, Garage Mode, suspend-to-RAM
12	OTA (ota)	Inactive slot glows while written, then slots swap or roll back	A/B updates
13	Security (security)	Chain of trust ticks green or breaks red; SecOC frame accepted or rejected	Secure boot, SecOC, Keystore
14	Multi-Display (zones)	A focus ring springs between seats	Occupant zones, MUMD
15	Car App Library (carapp)	Screens slide on push/pop; depth limit enforced	Templates, host

4 A 14-day learning plan

one chapter a day keeps the rejection away

Day	Read (book 02 unless noted)	Do in the app / code
1	00 Start Here + 02 ch.1 (the car as a computer network)	Build, run, play with Architecture
2	02 ch.2 AAOS architecture, Android Auto vs AAOS	Read CarApiVehicleDataSource.kt
3	02 ch.3 VHAL & properties	Decode 5 property IDs by hand, then check in the VHAL screen
4	02 ch.4 HVAC + ch.5 cluster	Drive in the Cluster screen; try shifting to P while moving
5	02 ch.6 Driver distraction	Move the speed slider; list what gets blocked
6	02 ch.7 Media + ch.8 Audio	Read CarMediaBrowserService.kt; trigger nav prompt during a call
7	02 ch.9 CAN, LIN, Ethernet, diagnostics	Run CanBusTest; do the arbitration exercise
8	02 ch.10 EVS/ADAS + ch.11 EV	Park Assist + EV screens
9	02 ch.12 Power + ch.13 OTA	Fire every power event; break an OTA
10	02 ch.14-16 multi-display, Car App Library, E/E architecture & SDV	Run the CarAppService on the DHU
11	03 Security (all)	Security screen; read SecOc.kt and SecureVault.kt
12	01 Project Structure	Add a new VHAL property to the catalog (exercise in book 01)
13	04 Interview Q&A; part A + B	Answer out loud and record yourself
14	04 part C + D (system design)	Whiteboard: design an AAOS media app end to end

RELAX

You do not need a real car or an AAOS device to follow this plan. Everything works on a phone emulator with the simulator. The AAOS emulator is a bonus for days 3-6.

5 Regenerating these PDFs

All five PDFs are generated from Python sources in `docs/source`. Edit the text, then run:

```
cd docs/source
pip3 install reportlab
python3 build_all.py      # writes ../00_Start_Here.pdf ... ../04_Interview_QA.pdf
```

BULLET POINTS

- Five books: start, structure, concepts, security, interviews.
- The app runs on phones (simulator) and AAOS (real Car API).
- Every lesson screen = live animation + Head First notes panel.
- Use `--es lesson <route>` to jump to any lesson.
- Follow the 14-day plan; say the answers out loud.